

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Домашняя/Лабораторная работа

Выполнила:

Голубева Мария

Группа
J3213

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г

Задача

Целью данной лабораторной работы было мигрировать ранее разработанное приложение (в рамках ЛР1 и ЛР2) на фреймворк Vue.js. В процессе работы было реализовано одностраничное веб-приложение (SPA), которое использует Vue.js для создания динамичного интерфейса, роутинг с помощью Vue Router, а также работу с внешним API для получения данных.

Ход работы

1. Подготовка проекта

Для начала был подготовлен проект на основе шаблона Vue 3 с использованием Vite, что позволило быстро настроить окружение для разработки и сборки. В проект были добавлены следующие основные зависимости:

- **Vue 3** — основной фреймворк для разработки компонентов и реактивного взаимодействия с пользователем.
- **Vite** — сборщик проекта, который значительно ускоряет процесс разработки благодаря быстрой сборке и перезагрузке.
- **Vue Router** — для маршрутизации между страницами приложения.
- **Axios** — для работы с внешним API.

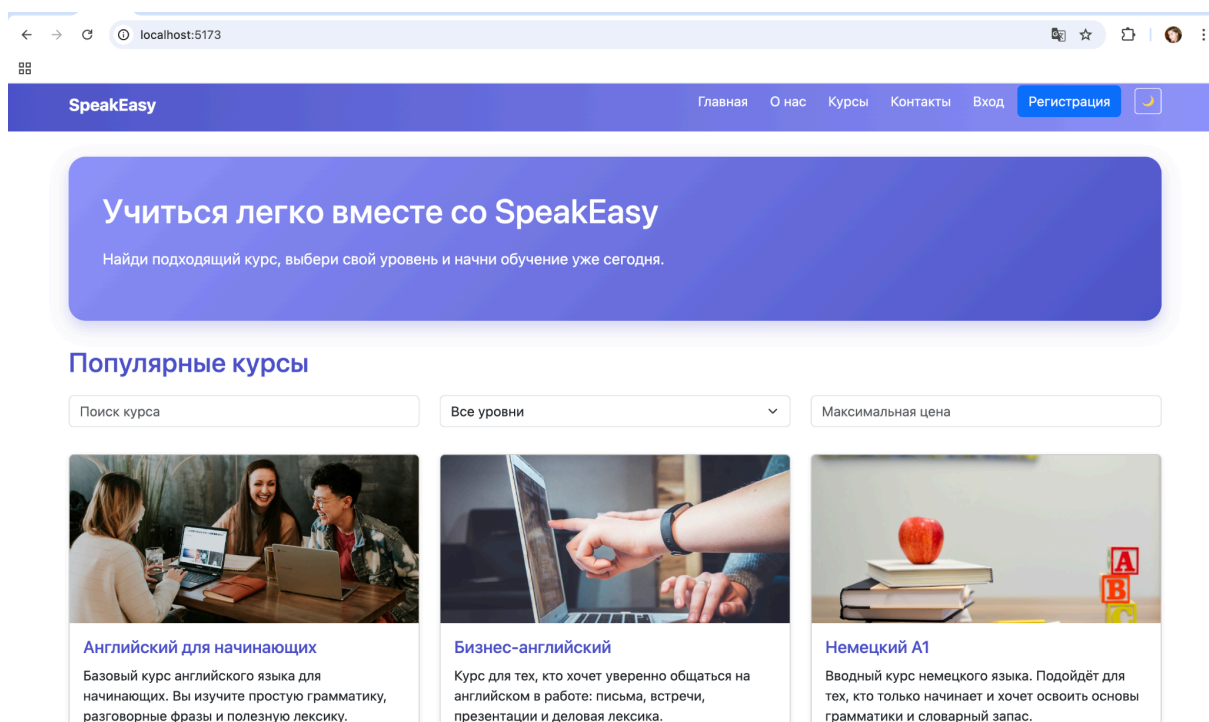
Проект был разделен на несколько компонентов для более чистой и структурированной архитектуры, что является примером применения компонентного подхода в Vue.

2. Компонентный подход и его преимущества

Компонентный подход в разработке — это методология, при которой интерфейс приложения разбивается на отдельные независимые компоненты. Каждый компонент может быть ответственен за одну конкретную задачу или часть интерфейса, что позволяет улучшить читаемость и поддержку кода. Преимущества покомпонентного подхода включают:

- **Модульность:** Каждый компонент имеет свою ответственность, и его можно разрабатывать и тестировать отдельно от других компонентов.
- **Переиспользуемость:** Компоненты могут быть переиспользованы в разных частях приложения, что ускоряет разработку и уменьшает дублирование кода.
- **Упрощение тестирования:** Легче тестировать отдельные компоненты, чем все приложение целиком.
- **Поддержка и расширение:** Изменения в одном компоненте не затрагивают другие, что упрощает поддержку и добавление новых функций.

В моем проекте это выражается в разделении приложения на компоненты, такие как **HeaderComponent.vue**, **FooterComponent.vue** и **CourseCard.vue**, которые могут быть переиспользованы на разных страницах, что улучшает структуру приложения.



3. Одностраничные приложения (SPA)

Одностраничное веб-приложение (SPA) — это тип веб-приложения, где все взаимодействие с пользователем происходит на одной странице. При использовании SPA весь интерфейс загружается один раз, и после этого

данные обновляются динамически, без перезагрузки страницы. Это достигается благодаря использованию JavaScript и роутинга.

Преимущества SPA:

- **Быстрая навигация:** Переход между страницами происходит без перезагрузки, что делает приложение более быстрым и отзывчивым.
- **Меньше запросов к серверу:** После первой загрузки, приложение не требует повторной загрузки всех ресурсов, что уменьшает нагрузку на сервер и ускоряет работу.
- **Лучший пользовательский опыт:** Так как приложение работает как единое целое, оно может обеспечивать более плавные и динамичные взаимодействия.

В нашем проекте используется Vue Router для реализации маршрутизации в SPA, что позволяет пользователю перемещаться по различным страницам приложения без полной перезагрузки страницы.

4. Применение Props и Emits в компонентах

В Vue.js **props** и **emits** используются для передачи данных между компонентами.

- **Props** — это механизм для передачи данных от родительского компонента к дочернему. Это позволяет дочерним компонентам получать данные и отображать их. Например, в компоненте **CourseCard.vue** передается объект `course`, который содержит информацию о курсе. В этом случае `course` — это пропс, передаваемый из родительского компонента.
- **Emits** — это механизм для отправки событий от дочернего компонента к родительскому. Например, если в компоненте нужно уведомить родительский компонент о каком-то действии (например, о нажатии кнопки или изменении состояния), используется `emit`-событие. В нашем проекте такие события могут быть использованы для уведомления родительских компонентов о действиях пользователей.

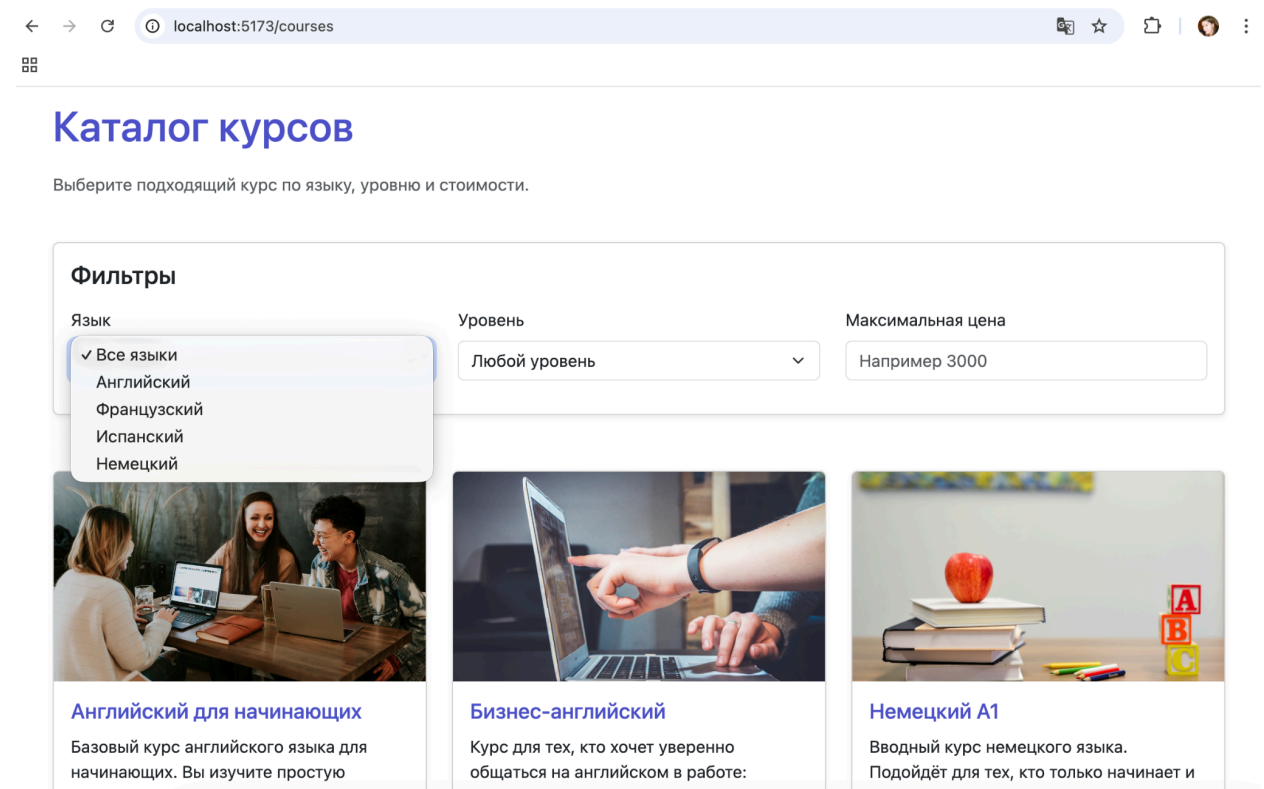
5. Composables и их использование

Composables — это функции, которые позволяют выделить повторяющийся функционал и использовать его в различных компонентах приложения. В Vue 3 с использованием Composition API можно создавать такие функции, чтобы избежать дублирования кода.

Преимущества использования composables:

- **Переиспользуемость кода:** Логика, которая используется в нескольких компонентах, может быть вынесена в composable и использована везде, где это необходимо.
- **Чистота кода:** Каждый composable выполняет одну задачу, что позволяет легче поддерживать и тестировать код.
- **Легкость в рефакторинге:** Изменения в логике можно внести в одном месте, и они автоматически применятся ко всем компонентам, которые используют данный composable.

В моем проекте примеры composables включают функции, такие как **useAuth** (для работы с авторизацией) и **useTheme** (для управления темой приложения). Эти функции используются в разных компонентах, например, для переключения темы в **HeaderComponent.vue** или для работы с состоянием пользователя.



Вывод

В результате выполнения лабораторной работы было создано одностраничное веб-приложение с использованием фреймворка Vue.js. Проект был успешно мигрирован с учетом требований к роутингу, работе с внешним API и разделен на компоненты. В ходе работы были получены навыки работы с компонентным подходом и Vue Router, а также освоены основные принципы работы с асинхронными запросами через axios.

Приложение теперь полноценно работает, позволяет пользователю просматривать курсы, авторизовываться и регистрироваться, а также управлять курсами через API.